Vertical Microprogrammed Control Unit

Design documentation for the vertical microprogramming module

1. Overview

The control unit tells the datapath what to do in every cycle. In this project it is microprogrammed: each machine instruction is implemented as a short routine of microinstructions stored in a read-only control memory (ROM). The horizontal unit uses a 42-bit encoded word containing control, ALU-function and sequencing fields. The vertical unit stores the same microprogram in a 16-bit word: a short micro-operation code plus an optional ALU function, and a small decoder expands the code back into the control signals.

The vertical unit is built from the horizontal microprogram. Every horizontal row is packed into a 16-bit word, then decoded again and compared with the original control information. A row that cannot be represented without losing a control signal is rejected. The two units are intended to be interchangeable: they present the same interface and produce equivalent control behaviour.

Parameter	Value
Control memory size	86 rows (addresses 0–85); 8-bit address, 255 reserved for END
Vertical word width	16 bits (horizontal: 42 bits)
Total vertical ROM	86 × 16 = 1,376 bits (172 bytes); horizontal 3,612 bits (451.5 bytes); 62% narrower
Control lines (Boolean)	18, plus a 5-bit ALU function (alu_op) = 23 control bits
Micro-operation codes	16 defined in a 5-bit field (32 possible); all 16 used by the current decoder
Microinstruction formats	2: Operate (bit 15 = 0) and Branch (bit 15 = 1)
Branch conditions	6: Z, !Z, N, C, V, GT
Opcode routines	30 machine instructions, plus ILLEGAL and TRAP rows and shared FETCH/DECODE front end

2. Control memory

2.1 Organization

The control memory is the vector vrom_ of 86 uint16_t words. A second vector cm_ holds the same rows after decoding (MicroOp structures); it is used for listings, validation and the control path utilities. The sequencer uses an 8-bit micro-PC. Address 0xFF (255) in a successor or target field means END, so an 86-row ROM leaves 169 rows available below the reserved END address.

Rows are grouped into routines laid out one after another:

- Row 0, ILLEGAL: a TRAP row used when an opcode has no routine entry; it raises the illegal-instruction exception.
- Rows 1–83: instruction routines and the separate hardware-fault TRAP routine. The hardware-fault entry is row 54; the remaining rows hold the 30 machine-instruction routines.
- Row 54, TRAP: target for hardware faults, including divide-by-zero and invalid memory access. This is separate from the ILLEGAL row at address 0.

- Rows 84–85: the shared front end, FETCH (IF) and DECODE (ID.DISPATCH). These are stored once and executed before every instruction.

A lookup table entries_[256] maps each opcode to the first row of its routine. In row 85, opcode_dispatch overrides the normal micro-ROM successor and selects routineEntry(opcode). Thus the shared front end is executed once per instruction: FETCH at 84, then DECODE/dispatch at 85, followed by the selected routine. For example, ADD follows 84 → 85 → 1 → 2 → 3 → END.

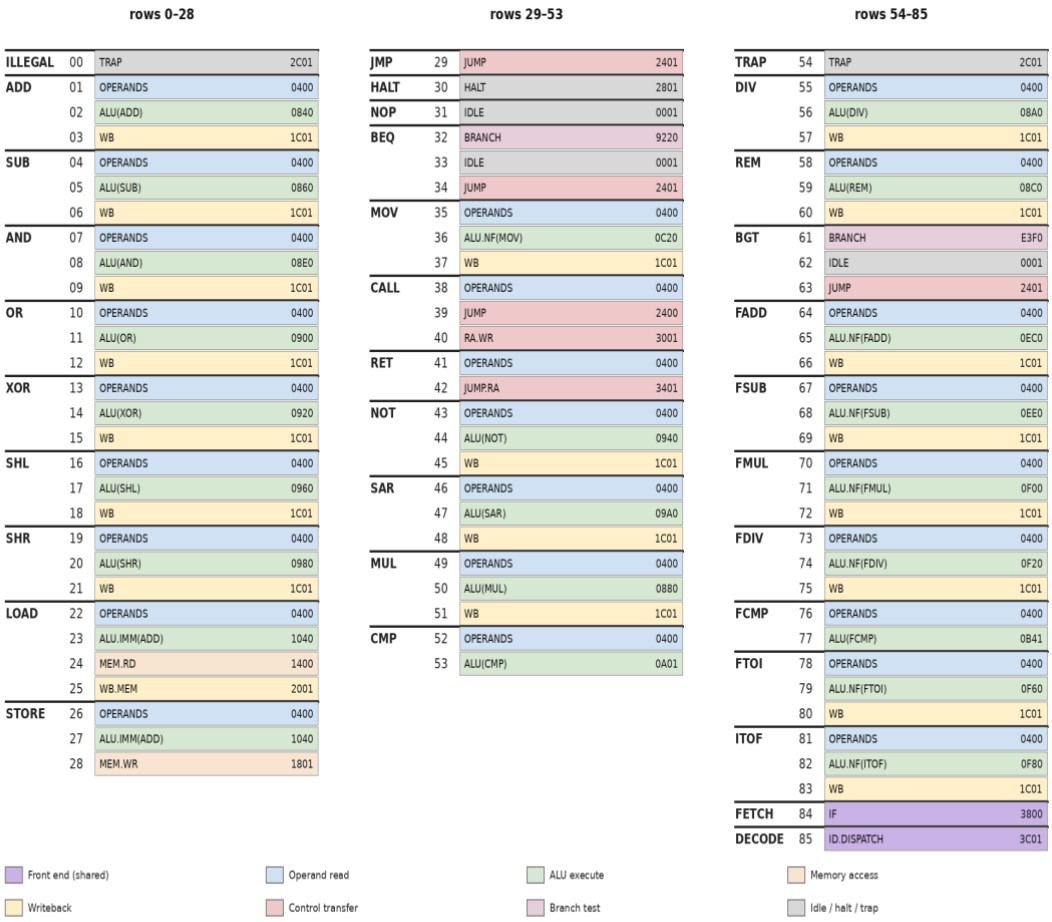


Figure 1. Updated vertical control memory map (86 rows × 16 bits). Heavy lines mark routine boundaries; hex values are 16-bit microwords.

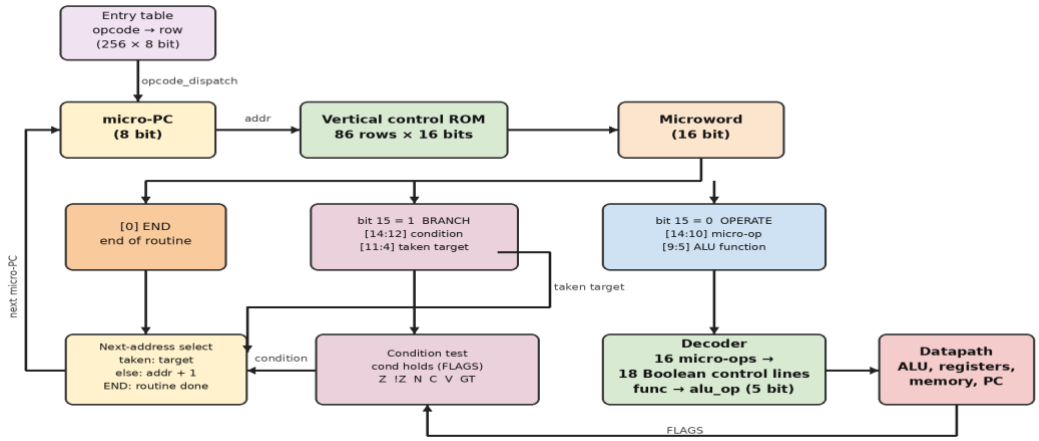
2.2 Sequencing (how a routine runs)

`VerticalControlUnit::step(op, flags, fault)` executes one micro-cycle:

- On the first cycle of an instruction it latches the entry address for `op` into the micro-PC. If that address is the ILLEGAL row, it records an illegal-instruction exception.
- It reads and decodes the row at the micro-PC and outputs its control word.
- If a fault is reported and the row's checker is active (ALU enable for divide-by-zero; addr_check for invalid memory access), the row's signals are cancelled, the exception is recorded and the micro-PC jumps to the TRAP row.
- Otherwise the successor is the branch target if the row is a branch and its condition holds, else the implicit next address (addr + 1), or END. At END the routine is done and the micro-PC resets for the next instruction.

Sequencing note: the standalone VerticalControlUnit::step(op, flags, fault) API starts at routineEntry(op) because its caller supplies the opcode. The processor pipeline instead starts each instruction at frontendEntry(): FETCH (row 84), then DECODE/dispatch (row 85), which selects the instruction routine. The controlPath() utility includes these shared front-end rows when displaying a complete instruction flow.

Figure 2. Vertical control-unit block diagram: 86-row ROM, 16-entry decoder, branch-condition evaluation and next-address selection.



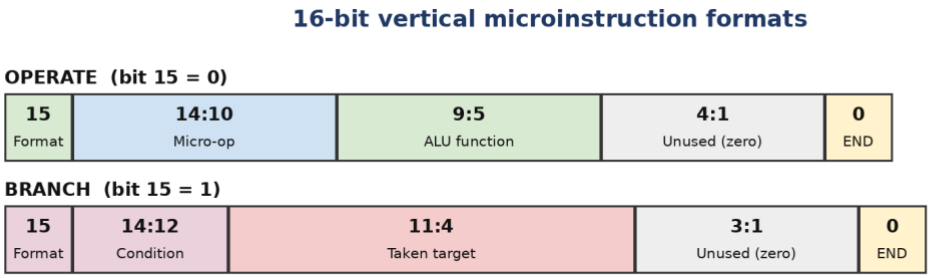
3. Control signals

The current ControlWord structure contains 18 Boolean control signals and one 5-bit field, alu_op, which selects the ALU function. These are 18 available Boolean fields; each micro-operation asserts only the signals it requires. The stack-related control lines from the earlier design are not present in the current ControlWord.

Signal	Group	Function	Used
ir_load	Front end	Load the fetched instruction word into the instruction latch	Yes
pc_inc	Front end	Advance the next-fetch PC by one instruction	Yes
instr_decode	Front end	Split the instruction into decoded fields	Yes
opcode_dispatch	Front end	Select the instruction's routine (overrides successor with entry address)	Yes
operand_read	Operands	Read source registers	Yes
alu_enable	ALU	Enable the ALU; alu_op selects the function	Yes
alu_src_imm	ALU	Use sign-extended immediate as second ALU operand	Yes
flag_update	ALU	Latch ALU result flags (Z, N, C, V, GT) into FLAGS	Yes
mem_read	Memory	Read data memory	Yes
mem_write	Memory	Write data memory	Yes
addr_check	Memory	Bounds-check the ALU-computed address before access; out of range raises a fault	Yes
reg_write	Writeback	Write the result to the destination register	Yes
mem_to_reg	Writeback	Select memory data as the value written back	Yes
pc_write	Control flow	Load the PC with a new value (jump, branch taken, call, return)	Yes
pc_src_ra	Control flow	Take the new PC from return-address register R14	Yes
ra_write	Control flow	Write the return address to R14 (CALL)	Yes
halt	System	Stop the processor	Yes

Signal	Group	Function	Used
trap	System	Raise exception/trap path (always together with halt)	Yes

4. Microinstruction format



Operate: [14:10] selects one of 16 micro-ops; [9:5] carries the ALU function when required.

Branch: [14:12] selects Z, !Z, N, C, V or GT; [11:4] is the taken address.

For both formats, bit 0 marks END. Unused bits must be zero; otherwise the encoding is non-canonical.

Figure 3. The two 16-bit vertical microinstruction formats.

Field	Bits	Meaning
Format	[15]	0 = Operate (drives datapath), 1 = Branch (tests FLAGS)
micro-op	[14:10]	Operate only: 5-bit code selecting one of the 16 decoder entries
func	[9:5]	Operate only: 5-bit ALU function, used only by ALU, ALU.NF and ALU.IMM (zero otherwise)
cond	[14:12]	Branch only: condition, 1 = Z, 2 = !Z, 3 = N, 4 = C, 5 = V, 6 = GT (0 and 7 invalid)
target	[11:4]	Branch only: 8-bit address of taken successor (0xFF = END)
END	[0]	Operate: last row of routine. Branch: not-taken path ends routine
unused	[4:1] / [3:1]	Always zero; tests verify canonical encodings

Sequencing rules that make the format compact:

- The successor is implicit: addr + 1, or END if bit 0 is set. A row whose horizontal successor is anything else cannot be packed. The DECODE row is a special dispatch case: opcode_dispatch overrides its normal successor and selects the opcode routine.
- A Branch row only tests FLAGS; it cannot also drive the datapath. Branch routines therefore use three rows: the condition test, the not-taken row (IDLE, END), and the taken row (JUMP, END).
- The not-taken path ends with an idle row. The processor continues at the sequential instruction address already advanced by FETCH; the branch microcode does not perform a PC write on this path.

Worked examples:

Row	Word	Decoding
2: ALU(ADD)	0x0840	0 | 00010 (ALU) | 00010 (func ADD) | 0000 | 0: asserts alu_enable, flag_update, alu_op = ADD
3: WB, END	0x1C01	0 | 00111 (WB) | 00000 | 0000 | 1: asserts reg_write; last row
32: BRANCH Z	0x9220	1 | 001 (Z) | 00100010 (target 34) | 000 | 0: if Z go to 34, else 33
84: IF	0x3800	FETCH: asserts ir_load and pc_inc
85: ID.DISPATCH, END	0x3C01	Decode instruction and dispatch to opcode routine

5. Decoder

The decoder is the table uopTable() (16 UopDef entries). It maps a 5-bit micro-op code to the set of ControlWord lines it asserts. If a micro-op takes a function, the func field is copied to alu_op. The table is checked at initialization to ensure that entries are in code order; otherwise logic_error is thrown. Codes 16–31 are undefined; decoding one yields a HALT row (fail-safe).

Code	Micro-op	Control lines asserted	Function	Used
0	IDLE	(none)	No operation; used by NOP and branch not-taken path	Yes
1	OPERANDS	operand_read	Read source registers	Yes
2	ALU	alu_enable, flag_update	Execute ALU function (func), update FLAGS	Yes
3	ALU.NF	alu_enable	Execute ALU function without updating FLAGS	Yes
4	ALU.IMM	alu_enable, alu_src_imm	ALU with immediate operand (address calculation)	Yes
5	MEM.RD	mem_read, addr_check	Bounds-checked data memory read	Yes
6	MEM.WR	mem_write, addr_check	Bounds-checked data memory write	Yes
7	WB	reg_write	Write ALU result to destination register	Yes
8	WB.MEM	reg_write, mem_to_reg	Write memory data to destination register	Yes
9	JUMP	pc_write	Load PC with jump/branch target	Yes
10	HALT	halt	Stop processor	Yes
11	TRAP	trap, halt	Exception: stop and flag trap	Yes
12	RA.WR	ra_write	Write return address to R14	Yes
13	JUMP.RA	pc_write, pc_src_ra	Return: PC from R14	Yes
14	IF	ir_load, pc_inc	Instruction fetch (shared front end)	Yes
15	ID.DISPATCH	instr_decode, opcode_dispatch	Decode and dispatch to instruction routine	Yes

The ALU function codes carried in func are the opcode values of the operation: MOV 1, ADD 2, SUB 3, MUL 4, DIV 5, REM 6, AND 7, OR 8, XOR 9, NOT 0xA, SHL 0xB, SHR 0xC, SAR 0xD, CMP 0x10, FADD 0x16, FSUB 0x17, FMUL 0x18, FDIV 0x19, FCMP 0x1A, FTOI 0x1B and ITOF 0x1C.

6. Microinstructions: the microprogram

Every machine instruction runs FETCH (84: IF, 0x3800) and DECODE (85: ID.DISPATCH, 0x3C01) first, then its own routine below. The routine rows are listed with their address and 16-bit microword. END is on the last row of each routine.

Instr.	Opcode	Rows	Microinstructions (word)	Function
NOP	00	31	IDLE 0001	Do nothing
MOV	01	35–37	OPERANDS 0400, ALU.NF(MOV) 0C20, WB 1C01	rd ← decoded source operand; flags unchanged
ADD	02	1–3	OPERANDS 0400, ALU(ADD) 0840, WB 1C01	rd ← rs1 + rs2; updates flags
SUB	03	4–6	OPERANDS 0400, ALU(SUB) 0860, WB 1C01	rd ← rs1 − rs2; updates flags
MUL	04	49–51	OPERANDS 0400, ALU(MUL) 0880, WB 1C01	rd ← rs1 × rs2
DIV	05	55–57	OPERANDS 0400, ALU(DIV) 08A0, WB 1C01	rd ← rs1 ÷ rs2; divide by zero traps
REM	06	58–60	OPERANDS 0400, ALU(REM) 08C0, WB 1C01	rd ← remainder of rs1 ÷ rs2
AND	07	7–9	OPERANDS 0400, ALU(AND) 08E0, WB 1C01	Bitwise AND
OR	08	10–12	OPERANDS 0400, ALU(OR) 0900, WB 1C01	Bitwise OR
XOR	09	13–15	OPERANDS 0400, ALU(XOR) 0920, WB 1C01	Bitwise XOR
NOT	0A	43–45	OPERANDS 0400, ALU(NOT) 0940, WB 1C01	Bitwise complement
SHL	0B	16–18	OPERANDS 0400, ALU(SHL) 0960, WB 1C01	Logical shift left
SHR	0C	19–21	OPERANDS 0400, ALU(SHR) 0980, WB 1C01	Logical shift right
SAR	0D	46–48	OPERANDS 0400, ALU(SAR) 09A0, WB 1C01	Arithmetic shift right
LOAD	0E	22–25	OPERANDS 0400, ALU.IMM(ADD) 1040, MEM.RD 1400, WB.MEM 2001	rd ← memory[base + imm]; bounds checked
STORE	0F	26–28	OPERANDS 0400, ALU.IMM(ADD) 1040, MEM.WR 1801	memory[base + imm] ← rd; bounds checked
CMP	10	52–53	OPERANDS 0400, ALU(CMP) 0A01	Subtract to set flags; no register write
JMP	11	29	JUMP 2401	PC ← target
BEQ	12	32–34	BRANCH Z 9220, IDLE 0001, JUMP 2401	If Z, PC ← target; else continue
BGT	13	61–63	BRANCH GT E3F0, IDLE 0001, JUMP 2401	If GT, PC ← target; else continue
CALL	14	38–40	OPERANDS 0400, JUMP 2400, RA.WR 3001	Save instruction PC + 1 in R14, then jump to target. RA.WR follows JUMP in the routine; the CPU uses the instruction's latched PC when writing R14.
RET	15	41–42	OPERANDS 0400, JUMP.RA 3401	PC ← R14

Instr.	Opcode	Rows	Microinstructions (word)	Function
FADD	16	64–66	OPERANDS 0400, ALU.NF(FADD) 0EC0, WB 1C01	Single-precision addition; flags unchanged
FSUB	17	67–69	OPERANDS 0400, ALU.NF(FSUB) 0EE0, WB 1C01	Single-precision subtraction; flags unchanged
FMUL	18	70–72	OPERANDS 0400, ALU.NF(FMUL) 0F00, WB 1C01	Single-precision multiplication; flags unchanged
FDIV	19	73–75	OPERANDS 0400, ALU.NF(FDIV) 0F20, WB 1C01	Single-precision division; flags unchanged
FCMP	1A	76–77	OPERANDS 0400, ALU(FCMP) 0B41	Float compare; sets flags, no register write
FTOI	1B	78–80	OPERANDS 0400, ALU.NF(FTOI) 0F60, WB 1C01	Convert floating-point value to integer
ITOF	1C	81–83	OPERANDS 0400, ALU.NF(ITOF) 0F80, WB 1C01	Convert integer value to floating point
HALT	1F	30	HALT 2801	Stop processor
ILLEGAL	any other	0	TRAP 2C01	Undefined opcode: illegal-instruction exception
TRAP	(fault)	54	TRAP 2C01	Target of divide-by-zero and invalid-memory faults

MOV, floating-point arithmetic and the conversion instructions use ALU.NF, so the architectural flags are left untouched. CMP and FCMP have no writeback row, so a branch can follow them. Rows 0 and 54 are two separate copies of the TRAP word: one for undefined opcodes and one for hardware faults.

7. Software implementation

The current header defines 42-bit horizontal words, 16-bit vertical words, 18 Boolean control lines and 16 micro-operation codes.

Function / type	Purpose
MicroOpCode (enum)	The 16 micro-op codes used by the decoder; microOpName(code) returns the mnemonic.
kHorizontalWordBits, kVerticalWordBits, kNumControlLines	42-bit horizontal words, 16-bit vertical words and 18 Boolean control lines, as defined by the current header.
uopTable()	Static decoder: UopDef { code, name, takes_func, control lines } for every micro-op; verifies table order.
lines({...})	Builds a ControlWord with the listed lines asserted.
lineBits(ControlWord)	Reduces a control word to a bit pattern of its lines, ignoring alu_op and sequencing.
packVertical(row, addr)	Encoder: checks successor, builds operate/branch word, verifies function and target field widths.
unpackVertical(word, addr)	Decoder: rebuilds a MicroOp from a word. Undefined encodings yield a HALT row.
canPackVertical(row, addr)	Returns whether a row can be encoded without loss.
VerticalControlUnit()	Builds the vertical ROM from the horizontal microprogram, decodes it back, and checks successors.
step(op, flags, fault)	Executes one micro-cycle, handles faults and restarts after END.
readRow(addr), reset()	ROM read/decode and sequencer reset.

Function / type	Purpose
routineEntry(op), frontendEntry(), trapEntry()	Entry addresses of instruction routines, shared front end and TRAP.
controlMemory(), verticalRom()	Access decoded rows and packed 16-bit ROM.
pendingException(), clearException(), routineDone(), microPC(), encoding()	State accessors; interface matches the horizontal ControlUnit.
dumpControlMemory()	Human-readable ROM listing with size comparison.

Supporting files that work with the vertical unit:

- control_path.cpp: frontendRows, routineRows, routineEnds and controlPath walk the ROM for either unit to show an instruction's complete flow.
- export_control_rom.cpp: generates docs/control_rom_vertical.txt and the horizontal listing; --check detects stale listings.
- tests/vertical_control_unit_test.cpp: contains checks for ROM sizes, exact microwords, canonical round-trips, horizontal/vertical equivalence, stage controls, lockstep execution and fault behavior.

8. Horizontal vs. vertical

	Horizontal	Vertical
Word width	42 bits	16 bits
Total ROM (86 rows)	3,612 bits (451.5 bytes)	1,376 bits (172 bytes)
Control line encoding	One bit per line, direct	Encoded micro-op, expanded by decoder
Successor addresses	Explicit next and branch fields	Implicit addr + 1 or END; branch target field
Branch plus datapath in one row	Allowed by horizontal MicroOp representation	Not allowed (branch rows test FLAGS only)
Cost	Wide ROM, no micro-op decoder	Narrow ROM, decoder needed; no timing model in simulator